Bilateral Receipt Admission: Cross-Organizational Action Provenance with Treaty-Bound DSSE

Anonymous Author(s)

Abstract

Existing supply-chain provenance attests how an artifact was built; cross-organizational agent invocations need provenance for how an action was admitted at the receiving kernel (the runtime admission engine inside each organization, not an OS kernel). SLSA, in-toto, Sigstore, and Rekor describe production; none attest that two parties jointly admitted an action under a treaty-bound predicate (the cross-organizational agreement that names participant identities, predicate scope, and lifetime) over identical canonical bytes including a continuation that links to receipt-graph state (the directed graph of admitted receipts and their causal links). We define a DSSE predicate type whose subject digest binds a tuple of treaty scope, ladder-intersection hash (the canonical hash of the joint mode-coverage table the two parties' constitutions agreed to), request and outcome digests, continuation hash (the linkage hash tying this admission to the prior receipt-graph state), and two receipt hashes under two named kernel keys, and a strict verifier with five rejection codes that distinguish noncanonical payloads, predicate-type mismatch, signer reuse, stale leases, and subject-digest mismatch. We implement the construction as a Rust runtime with a pre-dispatch admission hook that fails closed on federated-origin requests, with selective disclosure via BBS at presentation while Ed25519 remains authoritative for audit corpora. We demonstrate the construction with a three-vendor buyer-closure that replays both admitted and denied paths in the same canonical schema. The construction defeats sibling-treaty cross-receipt substitution, BBS stub-vs-real disambiguation, and error-message oracles by structural composition of the rejection codes with the canonical-bytes subject digest. The cryptographic primitive is self-standing: correctness rests on the conjunction of six gate predicates over canonical bytes, formalized in a Lean module whose only kernel axioms are propositional extensionality and choice. Broader polity, amendment, and refinement claims appear in the related polity-layer construction (anonymized for review, under separate submission) whose substrate this paper does not duplicate.

1 Introduction

Tool-using agents already behave like institutional actors, receiving delegated authority, calling tools across organizational boundaries, and emitting records that become accounting evidence. The weak point is provenance for cross-vendor action: existing supply-chain provenance (SLSA, in-toto, Sigstore, Rekor) attests how an artifact was *built*; agent invocations need provenance for how an action was *admitted at the receiving kernel*. No deployed primitive attests that two parties jointly admitted an action under a treaty-bound predicate over identical canonical bytes that include a continuation linking to receipt-graph state.

We close that gap with a single cryptographic construction. A bilateral DSSE envelope of predicate type `chio.bilateral-cosign-invocation.v1` (the strict bilateral-cosignature predicate, hereafter the Chiodos predicate) carries a subject digest computed over the canonical receipt body that binds treaty-scope hash (the canonical hash of the closed predicate set the two parties agreed to admit each other's actions under), ladder-intersection hash, request and outcome digests, continuation hash, and the two participating kernels' receipt hashes. A strict verifier accepts the envelope iff (i) both signatures verify against signer keys that are independent and pinned in the verifier-owned trust store (state local to the verifier, configured out-of-band, that names the keys, lease epochs, and treaty manifests the verifier trusts; nothing inside an envelope can mutate it); (ii) the payload is canonical JSON; (iii) the statement type is in-toto and the predicate type is the strict Chiodos profile; (iv) the leases referenced by the payload are fresh against the verifier's revocation epoch; and (v) the subject digest equals the canonical hash of the bound tuple. Five rejection codes distinguish each failure class so downstream audit can identify which constitutional precondition failed.

The bilateral-DSSE primitive is self-standing: its correctness is a property of the canonical-bytes envelope and the strict verifier alone, not of any higher-layer construction. Polity-scale claims (a polity is a closed receipt-admission boundary; cf. the broader formalization, anonymized for review), amendment refinement, and treaty-of-treaties composition appear in the related polity-layer construction (anonymized for review, under separate submission) whose substrate the present primitive does not duplicate.

The contributions are five falsifiable artifacts:

- A DSSE predicate type and canonical subject-digest binding for bilateral cross-kernel admission, with a five-code rejection taxonomy.
- A Rust runtime with a pre-dispatch admission hook that fails closed on federated-origin requests, plus a strict bilateral verifier.
- A three-vendor buyer-closure that replays admitted and denied paths in the same canonical schema.
- A negative-result characterization: sibling-treaty cross-receipt substitution, BBS stub-vs-real disambiguation, single-lane witness compromise, and error-message oracles are rejected by construction.
- A mechanically-checked accept-set witness in Lean 4 (`formal/lean4/Chio/Chio/Treaty/BilateralAccept.lean`) characterizing the verifier's accept relation as the conjunction of three abstract gates (issuer-key membership, kernel-key membership, scope-predicate denotation) that collapse the runtime's six-gate conjunction of Section 3 into a structural form Lean can mechanically check, with three derived corollaries (monotonicity under issuer-key extension, conjunctive scope decomposition, contrapositive trust-store membership), depending only on Lean kernel axioms with no `sorry` and no custom assumptions.

Two independent signatures over arbitrary bytes admit only that two parties signed the same bytes; treaty-binding admits that the two parties jointly authorized an action under a declared predicate set on a declared receipt-graph state, with rejection codes that distinguish each gate the verifier evaluated. The downstream auditor reading the receipt cannot confuse a cosigned envelope for a generic two-party signed artifact.

2 Receipt Admission as a Primitive

Signed-evidence systems decide one of two questions. Supply-chain transparency logs (Certificate Transparency, Rekor, in-toto, SLSA) answer how an artifact was built and whether a build-time claim was published to a public log [15, 21, 23, 24, 27]. Authorization systems (Cedar, OPA, capability OSes) answer whether a principal may perform an action inside a single trust boundary [7, 8, 14, 17]. Neither addresses the question agent systems force: whether two organizations jointly admitted a cross-vendor action under predicates each holds independently, in a form a third-party verifier can replay over canonical bytes.

The bilateral-receipt-admission primitive is the smallest construction that answers that question. It separates four concerns:

Local enforcement. Each kernel evaluates a local predicate set against a tool-call request. The predicate set may include capability attenuation [17], information-flow gates [28], or arbitrary local policy. The local decision is recorded in a canonical receipt body signed with the kernel's Ed25519 key.

Treaty-bound co-signing. When the action crosses an organizational boundary, both kernels independently sign a DSSE envelope whose subject digest binds the two receipts plus the treaty's scope, ladder, and continuation state. The envelope is admissible only when both signatures verify, the predicate type matches, and the canonical bytes are identical at both signers.

Strict rejection. A verifier accepts only envelopes that satisfy every precondition. The rejection codes are part of the protocol so downstream audit can identify the failing precondition.

Selective disclosure at presentation. BBS group signatures [3, 9, 16] project the receipt body for presentation to a third party (regulator, auditor, counterparty) without exposing every field. Ed25519 over canonical bytes remains authoritative for audit corpora; BBS is presentation-only.

The four concerns compose: local enforcement is independent of the cosigning protocol; cosigning is independent of selective disclosure; the verifier's rejection codes are independent of all three. A deployment can adopt them incrementally.

3 Predicate Schema and Strict Verifier

The bilateral admission envelope is a DSSE statement [21] carrying the predicate type `chio.bilateral-cosign-invocation.v1`. The payload type is the in-toto statement type, the statement type is in-toto v1, and the predicate body is a single record over canonical JSON [18] that binds, in one tuple: treaty id, treaty-scope hash, ladder-intersection hash, admission-report hash, continuation hash, request-canonical hash, outcome-canonical hash, local-receipt hash, remote-receipt hash, capability lease, governance receipt reference, and the two signer kernel ids. The DSSE subject is a single digest whose preimage is the canonical hash of that tuple; the envelope carries exactly two signature slices, one per kernel keyid.

This shape differs structurally from generic DSSE provenance. SLSA provenance and in-toto link statements [24, 27] bind a build pipeline to an artifact; their subject is the artifact digest and the predicate records build parameters. The bilateral profile binds a receipt-graph admission event to a treaty, with the subject digest taken over a tuple of hashes rather than an artifact, and with joint authorization intent expressed as two independent kernel signatures over identical canonical bytes. Without that intent record a receiver can verify that some party signed something, but cannot distinguish a shared action from two adjacent local logs that happen to name the same workflow.

The binding tuple. The subject digest is the canonical hash of a ten-field tuple, and each field commits to a distinct constitutional precondition. Figure 1 visualizes the binding-tuple decomposition. The *treaty-scope hash* pins the bilateral relationship in force (participant identities, named action classes, lifetime), so re-presenting the envelope under a different relationship is a hash miss rather than a re-routable token. The *ladder-intersection hash* pins the joint mode-coverage decision (which action class was evaluated at which mode under the intersected ladder), so a sender cannot retroactively claim a lower-intensity floor than the one both sides agreed to. The *admission-report hash* pins the receiving polity's intent: which predicates the receiver checks, which evidence it requires, and which fail-closed codes it has pre-declared. The *continuation hash* pins the linkage to receipt-graph state by naming the prior receipt this envelope extends, so the envelope cannot be re-grafted onto a different graph position without invalidating the digest. The *request hash* and *outcome hash* commit to the canonical input and output bytes separately, so selective disclosure can later reveal one without the other while preserving the binding. The *local-receipt* and *remote-receipt* hashes carry the sender's and receiver's signed receipt bodies, respectively; the pair is what distinguishes bilateral admission from a unilateral attestation. The *lease* field commits to a freshness window (a revocation epoch and an expiry), and the *signer kernel ids* commit to the two cosigners' identifiers, so an envelope cannot be reassigned to a different pair of signers post-facto.

Worked envelope. A valid envelope, with placeholder hashes, has the canonical shape (each hex field below is the bare lowercase output of SHA-256, truncated here with an ellipsis for display):

```
{ "_type": "in-toto-statement/v1",
  "predicateType":
    "chio.bilateral-cosign-invocation.v1",
  "subject": [ { "digest":
    { "sha256": "9f1c...e0a4" } } ],
  "predicate": {
    "treatyId":          "treaty:opus-alpha:2026",
    "treatyScopeHash":   "7b2a...c1e0",
    "ladderInterHash":   "8c4d...3f02",
    "admissionRptHash":  "a91e...204b",
    "continuationHash":  "5d22...8e07",
    "requestHash":       "11ff...6a91",
    "outcomeHash":       "33ee...b15d",
    "localReceiptHash":  "6677...d4cc",
    "remoteReceiptHash":"8899...02ab",
```

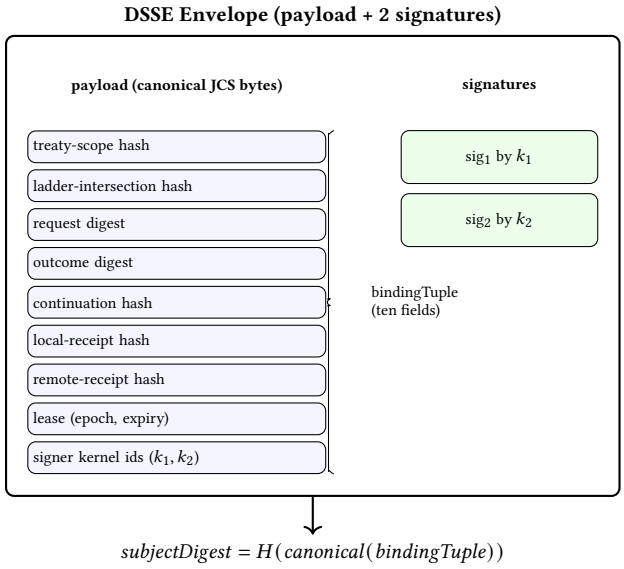

Figure 1: Bilateral DSSE envelope. The canonical payload binds a ten-field tuple, whose hash under canonical JCS encoding is the subject digest both kernel-key signatures cover.

```
    "lease": { "epoch": 412, "expiry": 1788000000 },
    "signerKids": [ "kid:opus#1", "kid:alpha#1" ] },
  "signatures": [
    { "keyid": "kid:opus#1",  "sig": "..." },
    { "keyid": "kid:alpha#1", "sig": "..." } ] }
```

The two `sig` slices are independent Ed25519 signatures over the identical canonical DSSE pre-authentication encoding of the payload bytes.

Canonical encoding of the binding tuple. The binding tuple is canonically encoded as an RFC 8785 JCS [18] object over a JSON record with ten named fields, one per constitutional precondition: `treatyScopeHash`, `ladderInterHash`, `admissionRptHash`, `continuationHash`, `requestHash`, `outcomeHash`, `localReceiptHash`, `remoteReceiptHash`, `leaseHash`, and `signerKidsHash`. Field names are fixed strings drawn from this enumeration; each field value is the hex encoding of a SHA-256 output, sixty-four lowercase ASCII characters with no `sha256:` prefix inside the binding object. The JCS rules force deterministic key ordering, minimal numeric form, and Unicode string normalization, so two implementations that agree on the input fields produce byte-identical encodings. The subject digest is then SHA-256(JCS(*bindingObject*)). The JCS layer supplies field-tag domain separation by construction: two distinct fields cannot collide on attacker-chosen content because their key strings differ in the canonical bytes that feed the digest. Raw concatenation, length-prefixed concatenation with field tags, and HKDF-Expand with field-tag info parameters are not used; the JCS choice inherits the deterministic-serialization guarantee already required for the outer DSSE envelope, and a second canonicalization surface would expose an additional attack surface without strengthening domain separation.

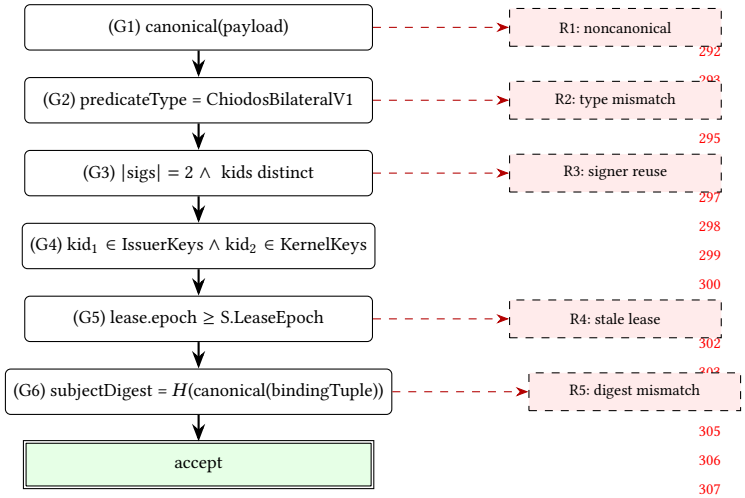

Figure 2: Verifier gate evaluation. Five named rejection codes distinguish noncanonical payloads, predicate-type mismatch (which bundles G4 trust-store membership), signer reuse, stale leases, and subject-digest mismatch.

Verifier accept set. Let the trust store $\mathcal{S} = (IssuerKeys, KernelKeys, LeaseEpoch)$ and let E be a bilateral envelope. The verifier accepts iff six gates hold in order; (G3) and (G4) below each group two atomic clauses that together express one structural property (signer distinctness, trust-store membership):

$$
\begin{aligned}
accept(\mathcal{S}, E) \iff\ & (\text{G1})\ canonical(E.payload) \wedge \\
& (\text{G2})\ E.predicateType = ChiodosBilateralV1 \wedge \\
& (\text{G3})\ \big(|E.sigs| = 2 \wedge \\
& \qquad E.sig_1.kid \neq E.sig_2.kid\big) \wedge \\
& (\text{G4})\ \big(E.sig_1.kid \in IssuerKeys \wedge \\
& \qquad E.sig_2.kid \in KernelKeys\big) \wedge \\
& (\text{G5})\ E.payload.lease.epoch \geq \mathcal{S}.LeaseEpoch \wedge \\
& (\text{G6})\ E.subjectDigest = \\
& \qquad H(canonical(E.payload.bindingTuple)).
\end{aligned}
$$

Each verified signature is over the same canonical payload bytes; the verifier never reaches (G1)-(G6) until both bytes-level signatures verify under the keyids declared in the signature slices. The predicate is the conjunction of six gates (G1)-(G6), evaluated left-to-right. Figure 2 renders the verifier gate sequence and the five rejection codes. Five of the six gates produce uniquely-named rejection codes; the trust-store-membership gate (G4) is bundled with predicate-type-mismatch in the deployed verifier, because a signer outside both allowlists fails (G2) before (G4) is reached whenever the predicate type is the strict Chiodos profile and the trust store enumerates only Chiodos-issuing keys.

noncanonical-payload. The payload bytes fail RFC 8785 canonicalization: re-ordered keys, non-minimal numeric forms, unnormalized strings, or trailing whitespace. The verifier rejects rather than re-canonicalizing because the signature is over a specific byte sequence. The attack class defeated is the *presentation attacker* who

serves different byte sequences to different verifiers, each of which re-canonicalizes locally and reaches a different decision while the upstream signature is undisturbed; the same channel also underwrites equivocation against transparency-log witnesses. Closing it at the verifier protects the canonical-bytes assumption that selective disclosure, replay-detection, and witness consistency all depend on.

predicate-type-mismatch. The DSSE statement carries a `predicateType` other than `chio.bilateral-cosign-invocation.v1`. The attack class defeated is *profile downgrade*: a sender substitutes a weaker predicate (an SLSA build provenance, a generic SCAI record, or a degenerate signed-JSON document) through the same envelope path, hoping the receiver's parser accepts any well-formed DSSE statement and applies bilateral semantics. Generic in-toto profiles do not bind admission-report and ladder-intersection hashes, so admitting them as bilateral collapses joint authorization to unilateral attestation while syntactic envelope checks still pass. Pinning the predicate string at verification rather than at parse time forces the substitution to forge a signature rather than to merely repackage one.

signer-reuse. The two signature slices carry the same keyid, or distinct keyids that resolve to the same operator under verifier-owned attribution metadata. The attack class defeated is the *confused-deputy cosignature*: one kernel produces both slices, possibly using two of its own key handles, and presents the envelope as bilateral consent. A single signer cannot produce two-party consent; party-independence is what distinguishes bilateral admission from a single kernel signing twice under different aliases. The keyid-equality test catches the trivial form; the operator-resolution test, parameterized by verifier-owned attribution metadata, catches the variant in which one operator runs two kernels under distinct keyids. The gate is conservative: a borderline pair is rejected rather than admitted, since admitting a sole signer destroys the property the envelope is meant to record.

stale-lease. The capability lease referenced by the payload predates the verifier's current revocation epoch, or its expiry has passed. The attack class defeated is *post-revocation replay*: an attacker holding an envelope whose underlying capability has since been revoked re-presents it to a verifier that has not retained per-action revocation entries, betting that the bilateral signatures remain bytes-valid. The lease binds the action's authority window; admitting a stale lease re-admits a window the issuer has already revoked and turns the receipt graph into an append-anything store. Lease freshness is a verifier-side test against state the verifier owns rather than a sender-asserted claim, which also rules out *epoch downgrade*: the comparison reads the verifier's epoch, not any field in the envelope. Intra-lease replay (resubmitting the same envelope within its still-current lease window) is not prevented at the verifier; it is a constitutional concern handled by the receipt-graph deduplication and continuation-hash discipline the related polity-layer construction develops (anonymized for review), not by the bilateral admission primitive.

subject-digest-mismatch. The DSSE subject digest is not the canonical hash of the binding tuple constructed from the payload's named fields. The attack class defeated is *sibling-treaty cross-receipt substitution*: an attacker takes a bytes-valid envelope authorized under treaty τ_1 and tries to re-present it under τ_2 by editing the binding tuple's treaty-scope or ladder-intersection field while leaving the upstream signatures in place. Any field-level edit produces a hash miss against the digest the signers committed to. The same gate also rules out *ladder-floor substitution*: the integrity check on the digest is structural, not field-by-field, so any tuple member that disagrees with the committed digest is rejected as a single failure rather than as a per-field diagnostic.

The taxonomy is intentionally coarse and is scoped to the post-signature gates: a signature-bytes failure denies the envelope before the six gates are evaluated and is not a member of the five rejection codes above. Each code names a conjunct of the admission predicate, not a structured failure report identifying which sub-field disagreed at the bit level. An adversary probing the verifier in the non-adaptive setting learns which gate first refused, not which inner field mismatched; the error surface leaks at most $\log_2 5 \approx 2.32$ bits per attempt, and that information is exactly what an auditor needs to identify which constitutional precondition failed. Against an adaptive adversary submitting envelopes in sequence, this bound holds per-attempt but not in aggregate: over N probes against a fixed target envelope, left-to-right gate evaluation reveals only the first refusing conjunct at each step, so an adversary who chains probes can localize the failing gate in $O(N)$ attempts and, with carefully constructed inputs, narrow the disagreement to a particular sub-field. This is the conjunct-localization that a non-error-message-oracle verifier should suppress; doing so requires uniform-time gate evaluation, which the present construction does not provide. Adaptive bounds against the gate-ordering oracle remain open.

4 Formal Sketch

The verifier of §3 admits a machine-checkable transcription of its accept relation in Lean 4. The transcription is not a deep theorem and is not load-bearing for the cryptographic security claim; it is an audit artifact that pins the informal predicate to a mechanical object so that the schema and the verifier cannot silently drift apart. Strip the runtime to three structures: a trust store $\mathcal{S}$ with separate issuer-key and kernel-key allowlists, a bilateral envelope E carrying a receipt id, a scope predicate, and two signature slices (one issuer-side, one kernel-side), and a denotational interpreter $[\![\cdot]\!]$ for the scope predicate against the receipt id. The accept relation is the conjunction

$$\begin{aligned} \mathit{accept}(\mathcal{S}, E) \;&= [E.\mathit{issuerSig.kid} \in \mathcal{S}.\mathit{issuerKeys}] \\ &\wedge [E.\mathit{kernelSig.kid} \in \mathcal{S}.\mathit{kernelKeys}] \\ &\wedge [\![E.\mathit{scope}]\!](E.\mathit{receiptId}). \end{aligned}$$

Signature byte validity is abstracted into trust-store membership: the trust store names keys whose signatures the deployed verifier has separately checked, and the Lean transcription inspects only the membership and scope conjuncts. The cryptographic primitives sit underneath this abstraction and are out of the Lean module's scope.

Three abstract gates vs. six runtime gates. The six runtime gates of §3 (canonical-payload, predicate-type, signer-distinctness, trust-store-membership, lease-freshness, subject-digest) collapse into

the three structural conjuncts above. Four of the runtime gates – canonical-payload, predicate-type, signer-distinctness, and lease-freshness – are assumed-sound at the byte-canonicalization and verifier-state layer that the trust-store conjuncts sit above: the trust store names keys whose signatures, predicate-type, and signer-distinctness checks the deployed verifier has separately discharged, and the lease-freshness check is a verifier-state precondition that yields to the trust-store conjuncts as a wrapper. The two load-bearing structural gates that remain visible to Lean are issuer-key membership and kernel-key membership, plus the scope-predicate denotation that the subject-digest gate witnesses at runtime. The transcription is therefore a structural projection, not a runtime equivalence.

Theorem 4.1 (`freestanding_accept_set_theorem`). *For every trust store $\mathcal{S}$ and bilateral envelope E,*

$$\begin{aligned} accept(\mathcal{S}, E) \iff & E.issuerSig.kid \in \mathcal{S}.issuerKeys \\ & \wedge E.kernelSig.kid \in \mathcal{S}.kernelKeys \\ & \wedge [\![E.scope]\!](E.receiptId) = true. \end{aligned}$$

The bi-conditional is definitional: *accept* is defined as exactly that Boolean conjunction lifted through `decide`, and the proof is one Lean tactic line that unfolds the definition and rewrites the resulting Boolean form. The statement therefore witnesses that the verifier's accept set decomposes into three abstract conjuncts – two trust-store-membership tests and one scope denotation – under the abstraction discussed above, with the canonical-payload, predicate-type, signer-distinctness, and lease-freshness runtime gates of §3 treated as preconditions of the trust-store conjuncts. Its work is to foreclose on accidental drift between the informal schema and the mechanical verifier as the predicate evolves; it does not constitute an independent proof that the cryptographic gates compose into a sound admission relation. The Lean source is self-contained at the module level: it imports the predicate-language type and its denotational interpreter, but takes no dependency on the wider polity model, anchor-quorum machinery, or treaty-intersection construction. A reviewer can audit the file end-to-end at `formal/lean4/Chio/Chio/Treaty/BilateralAccept.lean`.

Three corollaries follow definitionally and serve as scope-clarification artifacts rather than as independent theorems. The first, `accept_monotone_in_issuer_store`, names accept-set monotonicity in the trust store: if every issuer key in $\mathcal{S}$ is also in $\mathcal{S}'$ and the kernel-key allowlists agree, then $accept(\mathcal{S}, E) = true$ implies $accept(\mathcal{S}', E) = true$. The statement is the List-membership-monotonicity lemma over the issuer-key conjunct; its purpose is to record explicitly that adding a co-signing counterparty's key to the issuer allowlist never causes a previously-admitted envelope to be rejected. The second, `accept_conj_scope_decompose`, names accept-set conjunction over scope predicates: an envelope whose scope is $p \wedge q$ and which is accepted under $\mathcal{S}$ is also accepted under $\mathcal{S}$ when its scope is rebound to p alone, and likewise for q. This is the denotational identity $[\![p \wedge q]\!] = [\![p]\!] \wedge [\![q]\!]$ rephrased over the accept relation, useful for stating treaty-intersection refactors at the byte layer. The third, `accept_requires_issuer_key`, is the contrapositive of the first conjunct: if the envelope's issuer key id is not in $\mathcal{S}.issuerKeys$, then $accept(\mathcal{S}, E) = false$. All three are proved by unfolding *accept* and rewriting on the main bi-conditional.

Kernel-only axioms. Running Lean's `\#printaxioms` against `freestanding_accept_set_theorem` and each of the three corollaries reports only the kernel-level axioms *propext*, *Classical.choice*, and *Quot.sound*: the same trusted base every Mathlib development presupposes. No *sorry* placeholders appear, and the module introduces no custom axioms. This is a hygiene claim, not a security claim: every Lean development that avoids *sorry* and *axiom* declarations enjoys the same property, and the audit value is that any future strengthening of the module can be detected by an axiom-footprint diff against this baseline.

What the Lean module does not prove. The transcription is bounded by four explicit gaps. First, Ed25519 byte-level signature soundness: the verifier's signature check is an axiomatic primitive whose correctness lives outside the module, and the trust-store membership conjuncts presuppose that the deployed verifier has separately discharged it. Second, SHA-256 collision resistance and JCS injectivity: the subject digest is treated as a hash over canonical bytes, and any cross-field collision or canonicalization disagreement at the cryptographic layer is a structural assumption rather than a Lean theorem. Third, Rust-Lean refinement: the production verifier in `verify_chiodos_dsse_envelope` is not extracted from this module nor proven equivalent to it, and the two artifacts share no formal connection beyond the informal schema in §3. Fourth, polity-level admission: the module speaks only of a single bilateral envelope and does not address treaty intersection, single-amendment backward refinement, or the trajectory-invariant rule that closes constitutional ratchets; `amendment_admissible_iff_backward_refinement`, `treaty_admission_iff_predicate_intersection`, and `essential_preserved_chain` are discharged in the broader polity-level formalization (anonymized for review). The bilateral-DSSE primitive sits below those obligations: it is the byte-level admission gate the polity model presupposes, and the Lean module's job is to keep the gate's mechanical statement aligned with the schema rather than to derive the polity model's properties.

5 Implementation

The primitive lives in three Rust components: a runtime kernel that exposes a pre-dispatch admission hook, a federation crate that implements the strict bilateral DSSE verifier, and a selective-disclosure module that projects canonical receipt bytes into a BBS message vector for presentation. The wire format is the load-bearing artifact; the implementation is one realization of it.

Pre-dispatch admission hook. The runtime intercepts every tool-call before dispatch through a single hook whose only output is admit-or-deny. A request without a Chiodos admission context flows through the existing non-federated path; once a context is present, three behaviours become obligatory. First, the treaty reference is resolved by reading verifier-owned state alone, so any request-borne payload purporting to declare trust roots, peer directories, signing keys, or dynamic trust bundles is treated as adversarial input rather than configuration. Second, only the in-band DSSE evidence is verified, and only against signer sets that match the treaty's declared participants under the independence requirement on public keys. Third, on any denial or aborted continuation, reserved kernel state is released back to free, so that an attacker

watching a continuation get admitted then aborted cannot replay it under a fresh request.

Each of the three is a place where naive admission middleware degrades into advisory logging. Admitting federated origin without treaty context would bypass the primitive of §3 entirely; allowing request metadata to nominate the trust root would invalidate the verifier-owned-store assumption underwriting the freestanding accept-set theorem of §4; leaving continuation state allocated on denial would manifest as the replay edge the primitive is meant to close. The hook refuses each pattern as a structural matter, not as a runtime check an operator could disable.

Strict bilateral verifier. `verify_chiodos_dsse_envelope` (`crates/chio-federation/src/bilateral_dsse.rs:996`) is the cryptographic primitive at the heart of this paper: a total function from a candidate envelope and verifier-owned state to either one of the five rejection codes of §3 or an admission verdict that binds the two named cosigners to canonical bytes. Its internal gates split into two layers. The envelope layer enforces structural properties of the DSSE bytes themselves (payload-type equality against the bilateral statement type, signature count equal to two, canonical-JSON equivalence between the inner payload and its declared digest, predicate-type equality against `PREDICATE_TYPE_CHIODOS_BILATERAL`, single-subject presence, keyid-distinctness across the two signatures). The operational layer at `crates/chio-federation/src/bilateral_verifier.rs` then accepts only envelopes that already cleared the envelope layer and gates them against state the freestanding theorem leaves as trust-store inputs: peer key pinning, ladder-manifest freshness, revocation epoch, receipt resolution, request and tool-argument hashes, policy verdict agreement, capability lease, receipt-backed-class governance receipts, and treaty binding identifiers. The ordering is load-bearing: every byte-level check fires before the verifier consults state the sender cannot see, so a sender cannot satisfy a strict subset of the gates by tailoring bytes alone.

Each of the five rejection codes traces back to a specific gate position, and Table 1 maps the codes to their source locations. The taxonomy lives on the protocol surface, not in debugging output, because an audit consumer must be able to distinguish a wrong predicate type from a degenerate signer reuse from a stale lease from a sibling-treaty digest disagreement; each names a different constitutional precondition with a different remediation. The threat-model scope of the verifier is also worth naming explicitly. The construction binds two honest cosigners to canonical bytes against an arbitrary network adversary; a single actor who controls both signing keys, whether through cross-party key-custody compromise or through a single vendor operating both ends of a bilateral closure, sits outside the threat model. That case is treated as an operational assumption in §9 and is independent of every gate in the verifier.

Selective disclosure. The selective-disclosure module signs a BBS projection of the canonical receipt body and verifies derived disclosure proofs against an issuer-owned key registry [3, 16]. The Ed25519 signature over canonical receipt bytes remains authoritative for audit corpora; BBS is a presentation-only commitment that lets a holder reveal a subset of receipt fields to a verifier without exposing every field. The verifier pins the issuer key and the projection version, so the proof is not free-form redaction.

Table 1: Five rejection codes mapped to verifier source locations.

Rejection code	Verifier source
noncanonical-payload	canonical-bytes equality gate inside `verify_chiodos_dsse_envelope` (`bilateral_dsse.rs:996`)
predicate-type-mismatch	`predicate_type` equality against `PREDICATE_TYPE_CHIODOS_BILATERAL` inside `verify_chiodos_dsse_envelope` (`bilateral_dsse.rs:996`)
signer-reuse	keyid-independence gate and `require_unique_signature_keyids` inside `verify_chiodos_dsse_envelope` (`bilateral_dsse.rs:996`)
stale-lease	capability-lease freshness gate inside the operational verifier (`bilateral_verifier.rs`)
subject-digest-mismatch	binding-tuple subject equality gate inside the operational verifier (`bilateral_verifier.rs`)

Stub disclosure. One platform-specific stub the primitive does not depend on is worth naming. The macOS Endpoint Security path used for kernel-side telemetry is a stub: the integration surface compiles and tests pass against canned fixtures, but live Endpoint Security event delivery is not wired up. The bilateral-DSSE primitive, the admission hook, the federation crate, and the selective-disclosure module operate on canonical bytes and trust-store state regardless of whether kernel-side telemetry is live; the stub is named so a deployer planning kernel-event-driven receipts knows where the gap sits.

6 Evaluation

The primitive is exercised on a three-vendor closure: a buyer, two vendors, and a single bilateral admission between the vendors. The buyer owns the verifier context and the proof package; vendor receipts are not imported merely because they are signed but only when they bind treaty scope, ladder intersection, continuation, lineage, bilateral invocation, and strict DSSE evidence to one canonical subject digest. The closure is a generator: every run emits both signed positive packages (admitted vendor-to-vendor actions) and signed negative packages (denied actions) in the same canonical schema, so the buyer can replay every admission decision and every denial without privileged access to vendor internals.

Admitted path. A vendor-to-vendor request resolves to a treaty τ whose scope, ladder, and participant manifests are pinned in the buyer's verifier-owned trust store. The vendor kernels jointly produce a bilateral DSSE envelope whose subject digest binds τ's scope hash, the ladder-intersection hash, the request and outcome canonical hashes, the continuation hash, and the two vendor receipt hashes; the envelope carries one signature per vendor over identical canonical bytes. The admission hook resolves the treaty from the call site, consumes the continuation before dispatch, evaluates the six-gate verifier of §3, and admits the action. The receipt graph records one allow receipt per vendor plus the admission report; the buyer's negative-result audit later replays the same envelope against the same trust store and confirms admission.

Denied paths. Three denial fixtures cover the load-bearing rejection classes. (1) Sibling-treaty mismatch: a receipt issued under treaty T_A is replayed inside a request that resolves to sibling treaty

T_B; the subject digest fails because T_A's scope hash and ladder-intersection hash differ from T_B's, and the verifier returns subject-digest-mismatch. (2) Signer reuse: both DSSE signature slices carry the same vendor keyid, which fails the party-independence gate even though both signatures verify against pinned keys. (3) Stale lease: the capability lease referenced by the payload predates the buyer's revocation epoch; the lease-freshness gate fails before the digest gate is reached. In each case the negative-result package is signed, canonical, and shaped identically to a positive package: a downstream reviewer replays the canonical bytes and reconstructs both the admission attempt and the rejection code.

Dispatch latency. The pre-dispatch admission hook is reused from the Criterion bench for the allow path. On the baseline machine (Apple M1 Max, 10 cores, 32 GB RAM, Darwin 25.4.0) the median dispatch-admission time is p50 72.051 μs (Criterion 95% CI 71.745 μs to 72.367 μs), measuring the local-policy path that the federated admission hook composes with. The bilateral DSSE verification adds a constant cost on top (two Ed25519 verifies plus a SHA-256 over the canonical binding tuple), which dominates only when the kernel's local policy is trivially cheap.

Bench scope. The 72.051 μs figure measures the full admission-hook entry-point: capability decoding, scope and grant evaluation, the policy-lookup path, and the audit-log append the kernel emits on every allow verdict. The bilateral DSSE primitive proper (two Ed25519 verifications and one SHA-256 over the canonical binding tuple) is a fraction of this composite, and a verifier-only microbenchmark over `verify_chiodos_dsse_envelope` in isolation is deferred to a later harness. The number is therefore an upper bound on the bilateral-DSSE cost when the verifier composes with the kernel's local-policy path; it supports system-level statements about per-dispatch overhead but is not a cryptographic-primitive measurement.

Replay corpus stability. The replay gate runs a kernel-capability golden target across 50 fixtures spanning ten capability-side families (allow simple, allow metered, allow with delegation; deny expired, deny revoked, deny scope mismatch; guard rewrite; replay attack; tampered canonical JSON; tampered signature). Each fixture's expected verdict is canonicalized against a stored manifest; the reported property is manifest stability across machines rather than re-execution of the kernel on adversarial input. The reported result is 50 fixtures, manifest-stable across machines (test wall-time 1.07 s).

Bilateral-relevant fixture families. Four of the ten families bear directly on the cryptographic-primitive surface. The *replay-attack* family resubmits a previously admitted canonical envelope under a fresh request; the verdict denies on continuation-hash divergence, the receipt-graph state binding that the bilateral subject digest pins. The *tampered-canonical-JSON* family perturbs whitespace, key order, or numeric encoding under a fixed signature; the verdict denies on noncanonical-payload before any signature check, exercising the canonicalization gate of §3. The *tampered-signature* family flips one bit inside a DSSE signature slice under unchanged canonical bytes; the verdict denies on signature-bytes verification, the pre-gate precondition §3 names that must hold before any of the six gates is evaluated. The denial is therefore outside the five rejection-code taxonomy of §3 (which scopes its codes to the post-signature gates), and the fixture exercises the byte-level Ed25519 check rather than the gate conjunction. The *deny-scope-mismatch* family presents a receipt whose embedded treaty scope hash disagrees with the resolved treaty; the verdict denies on subject-digest-mismatch, the same rejection code the sibling-treaty closure of §7 exercises. The full ten-family enumeration belongs to the broader empirical evaluation (anonymized for review); a Chiodos-specific corpus covering predicate-type-mismatch, signer-reuse, and stale-lease fixtures is named as follow-up work alongside §9.

Treaty intersection. The treaty-intersection harness constructs two governance ladder manifests with N overlapping action classes, calls the ladder-intersection routine, and reports per-iteration percentiles for the cross-organizational admission cost. At N=1 the median is p50 131.67 μs with p99 194.00 μs; at N=10 p50 539.75 μs with p99 792.00 μs; at N=100 p50 4980.46 μs with p99 12138.25 μs. Growth is approximately linear in N because the implementation walks every allowed class and every participant manifest, hashes manifests, and builds per-class joined state; the p99 outlier at N=100 is reported, not smoothed. Treaty intersection is the closest measured analogue here to a cross-lane admission cost and bounds the per-dispatch overhead added on top of the local dispatch path of the preceding paragraph.

What is not measured here. Receipt sign and receipt verify latencies are out of scope: the existing Criterion scaffolds black-box constants, so reporting them would measure framework overhead rather than the primitive, and a dedicated harness is deferred to the broader empirical evaluation (anonymized for review). Anchor-inclusion latency across the Rekor, OpenTimestamps, Solana memo, and EVM event lanes is also unmeasured here; the lanes have uneven maturity and a single per-lane number would understate the multi-lane policy of §3. The measurements above bound the cryptographic-primitive surface, not the full polity-layer admission cost.

What the closure demonstrates. Three properties survive without the polity-layer framing of the related construction (anonymized for review). First, admission is byte-level reproducible: the buyer replays the canonical envelope and recovers either the same admission or the same rejection code. Second, denial is auditable in the same format as admission: a signed negative package is not a degraded log entry but a first-class artifact with the rejection code as a structured field. Third, the verifier-owned trust store is the boundary the construction defends: the admitted path and the three denied paths differ only in whether the canonical subject digest, the signer set, and the lease epoch agree with the buyer's trust state, and every disagreement is named by exactly one rejection code.

7 Attacks Defeated by Construction

Six classes of attack against cross-organizational receipt evidence are rejected structurally rather than detected. For each class the adversary capability, the verifier-visible state, the rejection that fires, and the residual risk are named.

Sibling-treaty cross-receipt substitution. The adversary holds a valid receipt admitted under treaty T_A and resubmits its DSSE envelope inside a request that resolves at the receiving kernel to sibling treaty T_B. The verifier reconstructs the subject digest against T_B's scope and ladder-intersection hashes; the digest disagrees with the one signed under T_A. The envelope verifies under the pinned vendor keys but is rejected on subject-digest-mismatch before any side effect. Residual risk: pinning T_A's scope hash into T_B's manifest is trust-store misconfiguration, not an envelope-level attack.

BBS stub-versus-real disambiguation. The adversary can mint a BBS proof over a chosen attribute vector but holds no Ed25519 signing key for either cosigner. Ed25519 is authoritative for admission; BBS is verified against the disclosed-attribute subset of the canonical body and is admissible only as a projection of an Ed25519-signed record already in the receipt graph. The rejection is structural: no Ed25519 record means no admissible projection, regardless of how cleanly the BBS proof verifies. Residual risk: a verifier configured to accept BBS presentations without an Ed25519 binding inverts the policy; the predicate type pins which signature is authoritative.

Single-lane witness compromise. The adversary controls one anchor lane (Rekor, OpenTimestamps, Solana memo, or an EVM event source) and can publish inclusion claims for chosen digests on that lane alone. The verifier evaluates the receipt's anchor set against the treaty's declared multi-lane witness policy. A receipt meeting the quorum outside the compromised lane still admits; a receipt witnessed only by the compromised lane fails the quorum predicate before subject-digest evaluation. Residual risk: a treaty naming a single-lane policy collapses to single-lane trust, which is a policy-layer choice.

Error-message oracles. An adversary probes the verifier with crafted envelopes to learn which sub-field disagrees with a target. The five rejection codes (noncanonical-payload, predicate-type-mismatch, signer-reuse, stale-lease, subject-digest-mismatch) are coarse: each names a conjunct of the admission predicate, not a structured failure report. The adversary learns which conjunct failed, not which byte disagreed, which keyid was reused, or which scope component diverged. Residual risk: richer diagnostics outside the rejection code (timing, byte hints, traces) reintroduce the oracle through observability.

Schema-version downgrade attack. The adversary holds a valid envelope minted under predicate version N and presents it to a verifier whose pinned predicate type is version $N+1$, or vice versa. The verifier compares the envelope's declared predicate type string against the resolved treaty's pinned type and rejects on predicate-type-mismatch before any signature check; the strict match in §3 excludes implicit forward or backward compatibility. Multi-version coexistence during a deployment migration window is out of scope for this primitive; the related polity-layer construction (anonymized for review) develops a schema-version-binding scheme that promotes predicate-type strict equality to a schema-rotation-aware acceptance relation, with explicit epoch boundaries that prevent the downgrade described here.

Constitutional ratchet at the policy layer. An attacker proposing an amendment that loosens admission for already-admitted history targets the predicate set itself rather than any individual envelope. This is out of scope at the envelope layer; the trajectory-invariant theorem (`essential_preserved_chain`), addressed in the related polity-layer construction (anonymized for review), obliges every amendment to refine the prior predicate set over admitted receipts, so no amendment can retroactively admit a previously denied class.

8 Related Work

Supply-chain provenance. The supply-chain lineage is closest in vocabulary but disjoint in admission semantics. In-toto link statements [27] provide *process* provenance: a signed record that a named build step transformed named inputs into a named output. The in-toto Attestation Framework v1.2.0 [13] generalises this to typed predicates over named subjects, separating envelope, statement, and predicate layers. SLSA [24] layers *level* provenance on top: the producer's pipeline is hardened to a numbered tier a verifier can require. Sigstore [23] and Rekor [22] log the resulting DSSE bundles [21]; IRONDICT [11] converges Rekor-style transparency logs with polynomial commitments, and distributed vector commitments [10] generalise the underlying binding to subvectors held across multiple machines. None of these attests *cross-organizational admission*: that the receiving kernel accepted this action under this treaty over identical canonical bytes including a continuation hash. A bilateral envelope can cite an in-toto v1.2.0 statement as build provenance for the binary referenced by its `requestHash`; the layers compose along the artifact-to-admission axis.

Transparency-service receipts and bilateral admission. The IETF SCITT architecture [2] is the closest standards-level comparator and admits a clean structural contrast. SCITT defines cross-organizational signed statements over named artifacts that an issuer registers with a transparency service; the service returns a COSE-encoded receipt [25] that witnesses inclusion in an append-only log, and the resulting envelope is a single-issuer statement countersigned by a separate service. Bilateral DSSE differs in the admission primitive: two organizations jointly sign the same canonical binding tuple at admission time, and the artifact a non-participant replays is a two-issuer joint accept rather than an issuer-plus-witness pair. The two regimes also separate temporally. SCITT receipts can post-date the statement; the transparency service is asynchronous to the issuer. Bilateral admission requires both verifications to converge on the same canonical bytes before either kernel emits a receipt, so the cosignature is itself the admission event. The regimes compose downstream: a bilateral receipt can be registered with a SCITT transparency service to gain log-rooted non-repudiation, but the joint accept that the receiving kernel observed is established by the bilateral cosignature, not by the subsequent log inclusion.

Property attestation. The closest formal lineage is property attestation; the delta sits at the layer the attestation describes. Integrity-measurement architectures [20] attest the *load-time state* of a software stack: a measured-boot chain commits the kernel and root filesystem to a TPM quote. Semantic and property-based attestation [12, 19] lift the surface from raw measurements to named

properties. Coker et al. [5] name the principles a trustworthy attestation must satisfy. The bilateral primitive attests the *runtime accept-set* of a verifier rather than a stack state; the accepted-receipt is a downstream consumer of property attestation, since a relying party can require TEE attestations over the cosigner kernels before admitting their envelopes.

Policy languages and verified authorization. Cedar [7, 8] establishes the closest Lean-plus-Rust industrial pattern. Cutler et al. prove a soundness theorem: any decision the Rust evaluator emits is derivable from the rule set under Cedar's Lean specification, with differential testing bridging the two implementations. The bilateral primitive proves the analogous claim about cross-organizational receipt admission rather than per-principal policy: the rule set is the Section 3 schema and the decision is the six-gate conjunction. The Lean module `BilateralAccept.lean` discharges the accept-set side; a Cedar-pattern refinement against a concrete Rust verifier is the next step. SAGA [26] distributes user-controlled access tokens from a Provider to constrain agent invocations; the Provider and the receiving kernel sit at different points on the same authorization path. Cremers et al. [6] prove a composition theorem in the Applied pi-Calculus that holds under dynamic-corruption attackers when the protocols satisfy an applied-pi disjointness requirement; the bilateral primitive composes two DSSE verifications at runtime over a single canonical byte string, a structurally simpler regime where the digest commitment substitutes for the symbolic disjointness condition.

Agent and tool isolation. The agentic-systems security literature names a closely-adjacent but topologically distinct boundary. IsolateGPT [28] enforces per-app isolation with information-flow gates inside one operator's runtime: the LLM and its tool calls execute under a single process trust boundary, and the gates protect tools from one another. Omega [1] proposes a TEE-rooted agentic runtime with declarative policy evaluation in the same single-operator setting. The bilateral primitive concerns the boundary *between* two organizations rather than *within* one; the artifact a non-participant can replay is the cross-organizational envelope, not an in-process gate decision. The two compose: an IsolateGPT-style gate is a local-policy predicate inside one party's Section 3 schema, and the cosigned envelope records what crossed.

Higher-layer formalization. A separate manuscript places the bilateral primitive inside a polity model and formalizes the higher-layer correctness statements as named Lean theorems: `treaty_admission_iff_predicate_intersection` characterizes treaty admission as the Boolean intersection of treaty scope, treaty constitution, and both polities' admission relations, and `essential_preserved_chain` characterizes amendment as backward refinement. The Section 4 accept-set witness is the byte-level gate those next-layer theorems quantify over; the present paper restricts to the lower layer because the accept-set statement carries on its own under any predicate language admitting denotational interpretation.

9 Limitations

Three load-bearing limits sit at the primitive layer.

No live cross-process federation. The multi-party setup runs in-process: two kernels share a clock and exchange canonical bytes through library calls rather than the network. Cross-process federation and partition reconciliation sit in the deployment trajectory the related polity-layer construction outlines (anonymized for review).

Single-vendor key custody. Bilateral DSSE carries party-independence only when the two signing keys are held by independent operators. If both kernels run under one custody domain the construction reduces to single-key signing, and operational discipline rather than cryptography carries the independence claim. A kernel-independence attestation primitive is named as future work.

Observability gap. The receipt log is the audit channel: every admission and denial appears as a canonical receipt with a rejection code. Live observability (denial-rate alerts, schema-mismatch fingerprinting, non-leaking telemetry) is not specified.

Side-channel attacks on the verifier. The strict verifier is treated as a constant-time idealization. In practice, timing leaks along the signature-verification path, cache-resident key material, and microarchitectural side effects during canonical-bytes comparison can leak information about the trust-store membership decision or the byte-level canonical encoding. The construction does not defend against side-channel attacks on the verifier's runtime; UncoreBleed [4] and contemporary side-channel literature against EdDSA verifiers and TEE-resident comparators indicate that a hardware-isolated deployment must independently mitigate against such leaks. The contribution here is the wire-format admission predicate, not a constant-time verifier, and the receipt-graph evidence captures the verifier's accept-reject verdict rather than the byte-level execution trace an adversary observes.

Malicious receiving kernel. The construction defends the sender's claim that a particular envelope was admitted under particular bytes; it does not defend against a receiver that controls its own trust store, suppresses denial receipts, or admits envelopes outside the declared predicate set. A receiving kernel that injects attacker-controlled keys into its allowlist can admit envelopes its declared policy should refuse; one that drops valid envelopes silently denies service. Such behavior is a constitutional violation rather than a cryptographic break: auditors with access to the receipt log detect the discrepancy by replay against canonical bytes, and a verifier that refuses to produce its receipt log at all is outside the scope of in-band detection. External assurance that a verifier evaluates the predicate it claims to evaluate is a kernel-attestation problem, addressed as future work.

Cryptographic-suite migration. Ed25519 signatures and SHA-256 subject digests are not post-quantum. A quantum-equipped adversary breaks Ed25519 signing-key recovery directly and weakens the SHA-256 collision-resistance margin; both impact the admission predicate at the binding-tuple and signature-slice layers. The related polity-layer construction develops a salted-hash plus PQC-signature migration path (anonymized for review), and the bilateral admission primitive inherits that path. Deployed verifiers will need re-anchoring against PQC-signed envelopes; the wire format does not currently version-tag for cipher-suite agility, and adding such

a tag (so a verifier can distinguish a v1 Ed25519/SHA-256 envelope from a successor profile without ambiguity) is part of the cryptographic-rotation work the related polity-layer construction outlines (anonymized for review).

Polity-level questions (citizenship, amendment, Hart-condition-(a) framing, trajectory-invariant theorems, sensor-state attestation) sit one layer above this primitive and are addressed in the related polity-layer construction (anonymized for review).

References

[1] Anonymous. 2025. Omega: A TEE-Rooted Agentic Runtime with Declarative Policy Evaluation. arXiv preprint. https://arxiv.org/html/2605.03213

[2] Henk Birkholz, Antoine Delignat-Lavaud, Cedric Fournet, Yogesh Deshpande, and Steve Lasker. 2025. An Architecture for Trustworthy and Transparent Digital Supply Chains. Internet-Draft draft-ietf-scitt-architecture-22 (AUTH48, Proposed Standard). https://datatracker.ietf.org/doc/draft-ietf-scitt-architecture/

[3] Dan Boneh, Xavier Boyen, and Hovav Shacham. 2004. Short Group Signatures. In *Advances in Cryptology - CRYPTO 2004*. Springer, Berlin, Heidelberg, 41–55. doi:10.1007/978-3-540-28628-8_3

[4] Decheng Chen, Zhi Zhang, Zhenkai Zhang, Xin Zhang, Yansong Gao, and Yi Zou. 2026. UncoreBleed: AEX-Free, High-Resolution, and Low-Noise Side-Channel Attacks on SGX Enclaved Execution. In *Proceedings of the 35th USENIX Security Symposium (Cycle 1)*. USENIX Association, Berkeley, CA, USA, 18 pages.

[5] George Coker, Joshua Guttman, Peter Loscocco, Amy Herzog, Jonathan Millen, Brian O'Hanlon, John Ramsdell, Ariel Segall, Justin Sheehy, and Brian Sniffen. 2011. Principles of Remote Attestation. *International Journal of Information Security* 10, 2 (2011), 63–81. doi:10.1007/s10207-011-0124-7

[6] Cas Cremers, Erik Pallas, and Aleksi Peltonen. 2026. Secure Protocol Composition under Dynamic Corruption: Scaling Up Symbolic Analysis for Real-World Security Properties. In *Proceedings of the 35th USENIX Security Symposium (Cycle 1)*. USENIX Association, Berkeley, CA, USA, 18 pages. https://eprint.iacr.org/2026/900

[7] Joseph W. Cutler, Craig Disselkoen, Aaron Eline, Shaobo He, Kyle Headley, Michael Hicks, Kesha Hietala, Eleftherios Ioannidis, John Kastner, Anwar Mamat, Darin McAdams, Matt McCutchen, Neha Rungta, Emina Torlak, and Andrew Wells. 2024. Cedar: A New Language for Expressive, Fast, Safe, and Analyzable Authorization. *Proceedings of the ACM on Programming Languages* 8, OOPSLA1 (2024), 123:1–123:30. doi:10.1145/3649835

[8] Craig Disselkoen, Aaron Eline, Shaobo He, Kyle Headley, Michael Hicks, Kesha Hietala, John Kastner, Anwar Mamat, Matt McCutchen, Neha Rungta, Bhakti Shah, Emina Torlak, and Andrew Wells. 2024. How We Built Cedar: A Verification-Guided Approach. arXiv:2407.01688. https://arxiv.org/abs/2407.01688

[9] ETSI. 2025. *Analysis of Selective Disclosure and Zero-Knowledge Proofs Applied to EUDI Wallet*. Technical Report TR 119 476-1 v1.3.1. European Telecommunications Standards Institute. https://www.etsi.org/deliver/etsi_tr/119400_119499/119476 01/01.03.01_60/tr_11947601v010301p.pdf

[10] Rui Gao, Huaqun Wang, Zhiguo Wan, and Yuncong Hu. 2026. Distributed Vector Commitments and Their Applications. In *Proceedings of the 35th USENIX Security Symposium (Cycle 1)*. USENIX Association, Berkeley, CA, USA, 18 pages.

[11] Hossein Hafezi, Alireza Shirzad, Benedikt Bünz, and Joseph Bonneau. 2026. Transparent Dictionaries from Polynomial Commitments. In *Proceedings of the 35th USENIX Security Symposium (Cycle 1)*. USENIX Association, Berkeley, CA, USA, 18 pages.

[12] Vivek Haldar, Deepak Chandra, and Michael Franz. 2004. Semantic Remote Attestation: A Virtual Machine Directed Approach to Trusted Computing. In *Proceedings of the 3rd Conference on Virtual Machine Research and Technology Symposium*. USENIX Association, Berkeley, CA, USA, 29–41. https://www.usenix.org/legacy/event/vm04/tech/haldar.html

[13] in-toto Project. 2024. in-toto Attestation Framework, v1.2.0. Specification v1.2.0, March 18, 2024. https://github.com/in-toto/attestation

[14] Gerwin Klein, Kevin Elphinstone, Gernot Heiser, June Andronick, David Cock, Philip Derrin, Dhammika Elkaduwe, Kai Engelhardt, Rafal Kolanski, Michael Norrish, Thomas Sewell, Harvey Tuch, and Simon Winwood. 2009. seL4: Formal Verification of an OS Kernel. In *Proceedings of the 22nd ACM Symposium on Operating Systems Principles*. ACM, New York, NY, USA, 207–220. doi:10.1145/1629575.1629596

[15] Ben Laurie, Emilia Messeri, and Rob Stradling. 2021. Certificate Transparency Version 2.0. RFC 9162. doi:10.17487/RFC9162

[16] Tobias Looker, Victoria Kalos, Mike Whitehead, and Ethan Bastian. 2026. The BBS Signature Scheme. Internet-Draft draft-irtf-cfrg-bbs-signatures-10. https://datatracker.ietf.org/doc/draft-irtf-cfrg-bbs-signatures/

[17] Mark S. Miller. 2006. *Robust Composition: Towards a Unified Approach to Access Control and Concurrency Control*. Ph. D. Dissertation. Johns Hopkins University. https://jscholarship.library.jhu.edu/bitstream/handle/1774.2/873/markm-thesis.pdf

[18] Anders Rundgren, Bret Jordan, and Samuel Erdtman. 2020. JSON Canonicalization Scheme (JCS). RFC 8785. doi:10.17487/RFC8785

[19] Ahmad-Reza Sadeghi and Christian Stüble. 2004. Property-Based Attestation for Computing Platforms: Caring About Properties, Not Mechanisms. In *Proceedings of the 2004 Workshop on New Security Paradigms (NSPW)*. ACM, New York, NY, USA, 67–77. doi:10.1145/1065907.1066038

[20] Reiner Sailer, Xiaolan Zhang, Trent Jaeger, and Leendert van Doorn. 2004. Design and Implementation of a TCG-Based Integrity Measurement Architecture. In *13th USENIX Security Symposium*. USENIX Association, Berkeley, CA, USA, 223–238. https://www.usenix.org/conference/13th-usenix-security-symposium/design-and-implementation-tcg-based-integrity-measurement

[21] Secure Systems Lab. 2026. Dead Simple Signing Envelope. https://github.com/secure-systems-lab/dsse

[22] Sigstore Project. 2026. Rekor: Software Supply Chain Transparency Log. https://github.com/sigstore/rekor

[23] Sigstore Project. 2026. Sigstore Security Model. https://docs.sigstore.dev/about/security/

[24] SLSA Framework. 2026. Supply-chain Levels for Software Artifacts Specification. https://slsa.dev/spec/latest/

[25] Orie Steele, Henk Birkholz, Antoine Delignat-Lavaud, and Cedric Fournet. 2025. COSE (CBOR Object Signing and Encryption) Receipts. RFC 9942 (AUTH48, Proposed Standard). doi:10.17487/RFC9942

[26] Georgios Syros, Anshuman Suri, Jacob Ginesin, Cristina Nita-Rotaru, and Alina Oprea. 2026. SAGA: A Security Architecture for Governing AI Agentic Systems. In *33rd Network and Distributed System Security Symposium (NDSS)*. Internet Society, Reston, VA, USA, 1–19. https://www.ndss-symposium.org/ndss-paper/saga-a-security-architecture-for-governing-ai-agentic-systems/

[27] Santiago Torres-Arias, Hammad Afzali, Trishank Karthik Kuppusamy, Reza Curtmola, and Justin Cappos. 2019. in-toto: Providing Farm-to-Table Guarantees for Bits and Bytes. In *28th USENIX Security Symposium*. USENIX Association, Berkeley, CA, USA, 1393–1410. https://www.usenix.org/conference/usenixsecurity19/presentation/torres-arias

[28] Yuhao Wu, Franziska Roesner, Tadayoshi Kohno, Ning Zhang, and Umar Iqbal. 2025. IsolateGPT: An Execution Isolation Architecture for LLM-Based Agentic Systems. In *32nd Network and Distributed System Security Symposium (NDSS)*. Internet Society, Reston, VA, USA, 1–18. https://www.ndss-symposium.org/ndss-paper/isolategpt-an-execution-isolation-architecture-for-llm-based-agentic-systems/